Rozdział 2

Instrukcje: Język komputera

Lista instrukcji

- Repertuar instrukcji komputera
- Różne komputery mają różne listy instrukcji
  - Ale wiele ich cech jest wspólnych
- Wczesne komputery miały bardzo proste listy instrukcji
  - Uproszczona implementacja
- Wiele współczesnych komputerów także ma proste listy instrukcji

Lista instrukcji ARM'a

- Używana w alternatywnych wersjach rozdziałów 2 i 3 książki
- W podstawowej wersji książki odniesieniem jest architektura Stanford MIPS (www.mips.com)
- Najpopularniejszy procesor 32-bitowy, głównie w komputerach zagnieżdżonych
  - Zastosowania w elektronice konsumenckiej, sprzęcie sieciowym/pamięciowym, kamerach, drukarkach, …
- Typowa dla wielu nowoczesnych ISA
  - Pełne porównanie w dodatku E (Appendix E)

Operacje arytmetyczne

- Dodaj i odejmij, trzy operandy
  - Dwa źródłowe i jeden przeznaczenia

```
ADD a, b, c  # a dostaje b + c
```

- Wszystkie operacje arytmetyczne mają taką postać
- *Zasada projektowania 1:* Regularność sprzyja prostocie
  - Regularność upraszcza implementację
  - Prostota prowadzi do większej wydajności przy niższym koszcie

Przykład

- Kod w C:

```
f = (g + h) - (i + j);
```

- Skompilowany kod ARM'a:

```
ADD t0, g, h   ; tymcz. t0 = g + h
ADD t1, i, j   ; tymcz. t1 = i + j
SUB f, t0, t1  ; f = t0 - t1
```

Rejestry jako operandy

- W instrukcjach arytmetycznych operandami są rejestry (register operands)
- ARM ma 16 rejestrów 32-bitowych
  - Do przechowywania często używanych danych
  - Numerowane od 0 do 15: r0, r1, …, r15
  - 32-bitowa dana to „słowo” (word)

- *Zasada projektowania 2:* Mniejsze jest szybsze
  - Porównaj z pamięcią główną: miliony lokalizacji

Przykład, c.d.

- Kod C:

```
f = (g + h) - (i + j);
```

  - f, …, j w rejestrach r0, r1, r2, r3, r4, wartości tymczasowe t0, t1 w r5 i r6

- Skompilowany kod ARM'a:

```
ADD r5, r0, r1
ADD r6, r2, r3
SUB r4, r5, r6
```

Operandy w pamięci

- Złożone dane są przechowywane w pamięci głównej (memory operands)
  - Tablice, struktury, dane dynamiczne
- W celu zastosowania operacji arytmetycznych
  - Załaduj wartości z pamięci do rejestrów
  - Prześlij wynik z rejestru do pamieci
- Pamięć jest adresowana bajtowo
  - Każdy adres wskazuje 8-bitowy bajt
- Adres słowa (word) nie jest dowolny
  - Musi być wielokrotnością 4 (aligment restriction)
- ARM należy do Little Endian'ów
  - Najmłodszy bajt ma najniższy adres w słowie
  - Big Endian: Najstarszy bajt ma najniższy adres

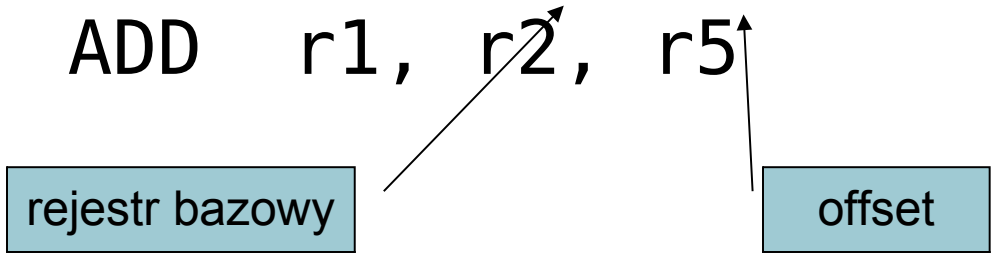
Przykład 1

- Kod C:

  `g = h + A[8];`

  - g w r1, h w r2, adres bazowy A w r3
- Skompilowany kod ARM'a:
  - Indeks 8 oznacza offset 32
    - Bo 4 bajty na słowo

```
LDR  r5, [r3,#32]  ; tymcz. r5
ADD  r1, r2, r5
```



Przykład 2

- Kod C:

```
A[12] = h + A[8];
```

  - h w r2, adres bazowy A w r3
- Skompilowany kod ARM'a:
  - Indeks 8 oznacza offset 32

```
LDR  r5, [r3,#32]   ; load word
ADD  r5, r2, r5
STR  r5, [r3,#48]   ; store word
```

Rejestry – pamięć

- Dostęp do rejestrów jest szybszy niż do pamięci
- Działania na pamięci wymagają instrukcji transferu danych (load, store)
  - Trzeba wykonać więcej instrukcji
- Kompilator powinien przechowywać zmienne przede wszystkim w rejestrach
  - „Zrzut” do pamięci tylko rzadziej używanych zmiennych
  - Optymalizacja zawartości rejestrów jest ważna!

Operandy natychmiastowe

- Stałe specyfikuje się w wariantach instrukcji

```
ADD r3, r3, #4   ; r3 = r3 + 4
```

- *Zasada projektowania 3:* Czyń typowy przypadek szybkim
  - Małe wartości stałe są często używane
  - Operandy natychmiastowe (immediate operands) pozwalają unikać operacji ładowania z pamięci

Liczby binarne bez znaku

- Liczba n-bitowa (binary unsigned integer)

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Zakres: 0 do $+2^n - 1$
- Przykład
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Zakres przy 32 bitach
  - 0 do +4,294,967,295

Uzupełnienie do 2

- Znakowa liczba n-bitowa (2s-complement)

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Zakres: -2^{n-1} do $+2^{n-1} - 1$
- Przykład
  - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
    $= -1\times2^{31} + 1\times2^{30} + \ldots + 1\times2^2 + 0\times2^1 + 0\times2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Zakres przy 32 bitach
  - –2,147,483,648 do +2,147,483,647

Uzupełnienie do 2

- Bit 31 jest bitem znaku
  - 1 dla liczb ujemnych
  - 0 dla liczb nieujemnych
- $-(-2^{n-1})$ nie ma reprezentacji
- Liczby nieujemne mają tą samą reprezentację bezznakową, jak i w uzupełnieniu do 2
- Niektóre szczególne liczby
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Najbardziej ujemna: 1000 0000 … 0000
  - Największa: 0111 1111 … 1111

Zmiana znaku

- Zaneguj i dodaj 1
  - Negowanie oznacza 1 ⟶ 0, 0 ⟶ 1

$$x + \bar{x} = 1111...111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Przykład: zmień znak +2
  - $+2 = 0000\ 0000\ \ldots\ 0010_2$
  - $-2 = 1111\ 1111\ \ldots\ 1101_2 + 1$
    $= 1111\ 1111\ \ldots\ 1110_2$

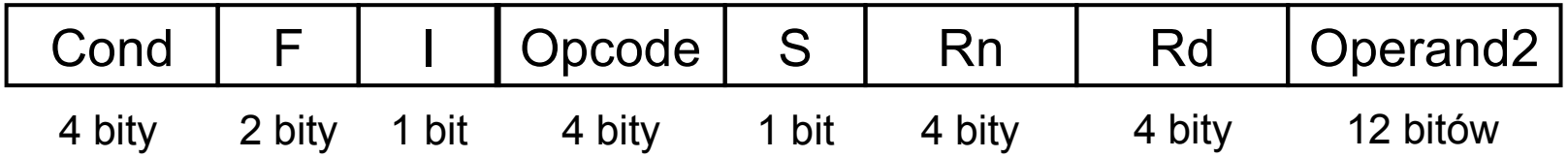
Rozszerzanie znaku

- Zwiększanie liczby bitów reprezentujących liczbę
  - Przy zachowaniu jej wartości
- W liście instrukcji ARM
  - `ADD, transfery danych`: rozszerzają pole natychmiastowe
  - `skoki`: rozszerzają przesunięcie (displacement)
- Powtarzaj bit znaku w lewo
  - W liczbach bez znaku rozszerzaj zerami
- Przykłady: od 8- do 16-bitów
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110

Reprezentowanie instrukcji

- Instrukcje są kodowane binarnie
  - Określa się to „kod maszynowy” (machine code)
- Instrukcje ARM’a
  - Kodowane jako słowa 32-bitowe (też tryb 16-bitowy - Thumb)
  - Mała liczba formatów kodów operacji (operation code, opcode), numery rejestrów, …
  - Regularność!

Formaty instrukcji ARM'a



- Pola instrukcji przetwarzania danych (DP)
  - Opcode, S: kod operacji, ustawianie znaczników
  - Rn: numer pierwszego rejestru źródłowego
  - Rd: numer rejestru przeznaczenia
  - Operand2: Drugi operand źródłowy
  - I: I=1: operandem jest wartość pola Operand2
  - Cond: warunek wykonania instrukcji
  - F: rodzaj formatu instrukcji (DP: kod 00)

Przykłady

Cond	F	I	Opcode	S	Rn	Rd	Operand2

```
ADD r5, r1, r2
```

14	0	0	4	0	1	5	2

1110	00	0	0100	0	0001	0101	000000000010

$$11100000100000010101000000000010_2 = e0815002_{16}$$

```
ADD r3, r3, #4
```

14	0	1	4	0	3	3	4

1110	00	1	0100	0	0011	0011	000000000100

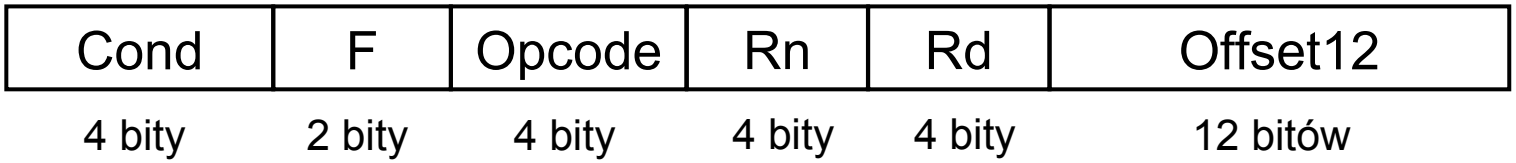
Kod heksadecymalny

- System szesnastkowy
  - Zwarta reprezentacja ciągów bitów
  - 4 bity na cyfrę szesnastkową

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

- Przykład: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

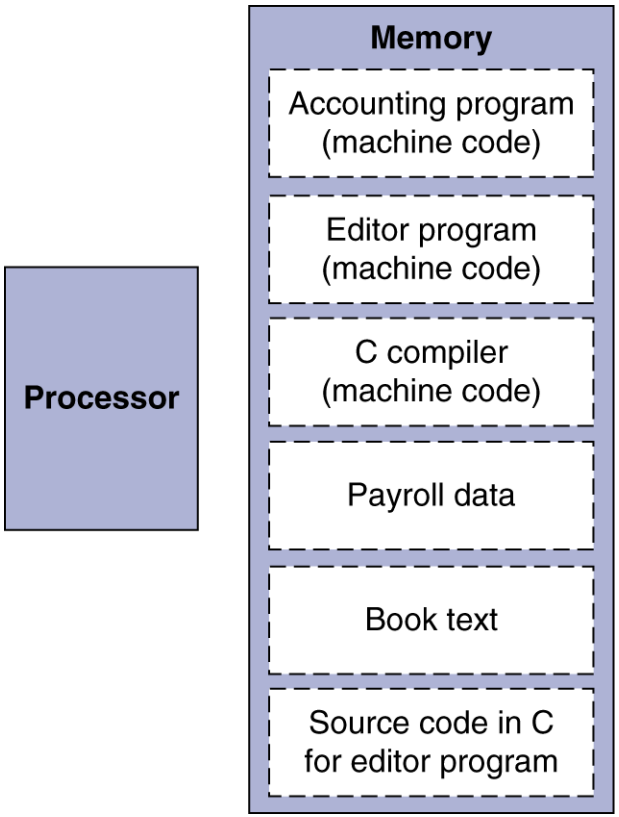
Formaty instrukcji ARM'a



- Pola instrukcji transferu danych (DT)
  - Rd: rejestr źródłowy lub przeznaczenia
  - Offset12: -2^{11} do $+2^{11} - 1$
  - Adres: offset12 dodany do adresu bazowego w Rn
  - F = 1, czyli 01_2
- *Zasada projektowania 4:* Dobry projekt wymaga dobrych kompromisów
  - Wielość formatów utrudnia dekodowanie, ale pozwala zachować stałą 32-bitową długość słowa instrukcji
  - Formaty powinny być do siebie podobne

Stored Program Computers

The BIG Picture



- Instrukcje reprezentowane binarnie, jak dane
- Instrukcje i dane przechowywane w pamięci
- Programy mogą przetwarzać programy
  - np., kompilatory, linkery, …
- Kompatybilność binarna zezwala skompilowanym programom działać na innych komputerach
  - Standardowe ISA

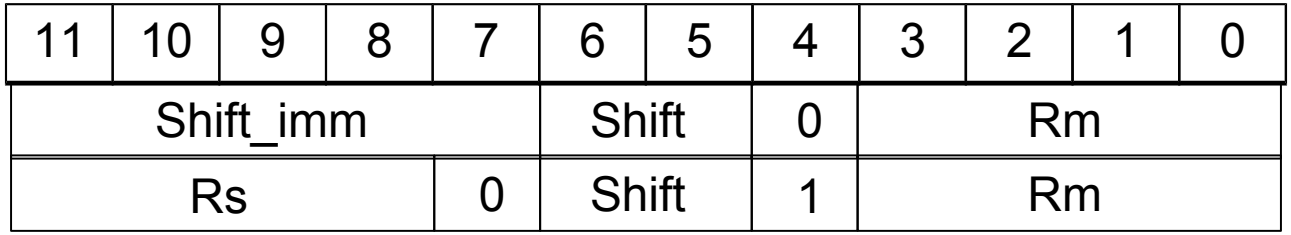
Operacje logiczne

- Instrukcje do manipulacji bitowej

Operacja	C	Java	ARM
Przesuń w lewo	<<	<<	LSL
Przesuń w prawo	>>	>>>	LSR
Bitowy AND (I)	&	&	AND
Bitowy OR (LUB)	|	|	ORR
Bitowy NOT (NIE)	~	~	MVN

- Używane do usuwania i wprowadzania grup bitów do słowa

Instrukcje przesunięcia

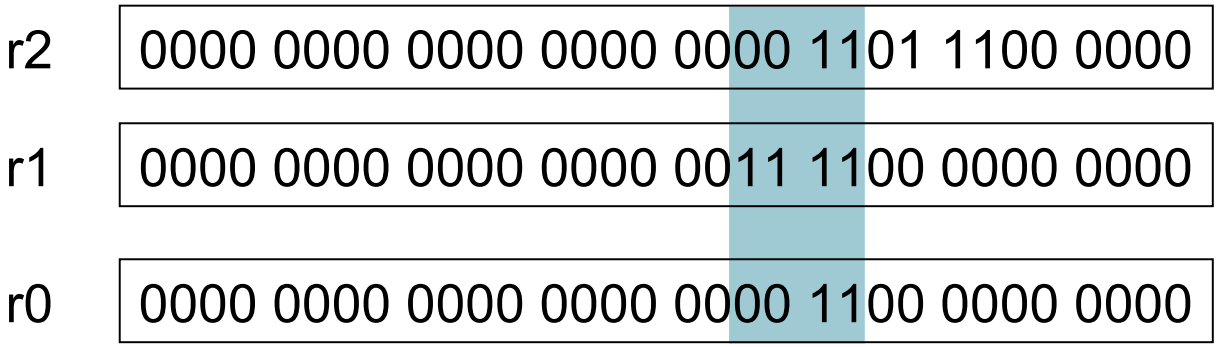


- Pole Operand2: format zależy od bitu 4
- Dla wszystkich instrukcji DP!
  - `Shift`: typ przesunięcia: 0 – logiczne w lewo
  - `Shift_imm`: wielkość przesunięcia
  - `Rs`: rejestr wskazujący wielkość przesunięcia
  - `Rm`: drugi operand instrukcji, zawartość tego rejestru ulega przesunięciu przed operacją
- Przykład: ADD r5, r1, r2, LSL #2

Operacja AND

- Używana do maskowania bitów słowa
  - Wyzeruj bity poza wybranymi

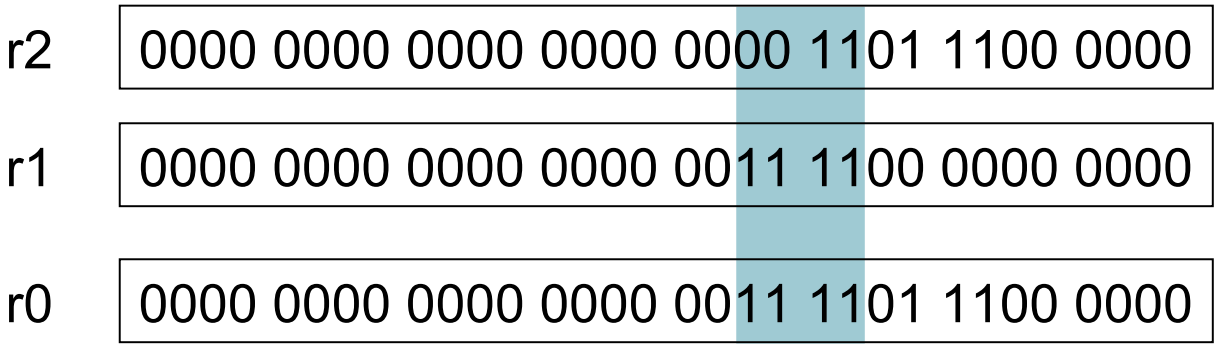
```
AND r0, r1, r2
```



Operacja OR

- Przydatna do włączania bitów do słowa
  - Ustaw wybrane bity bez zmiany innych

```
ORR r0, r1, r2
```

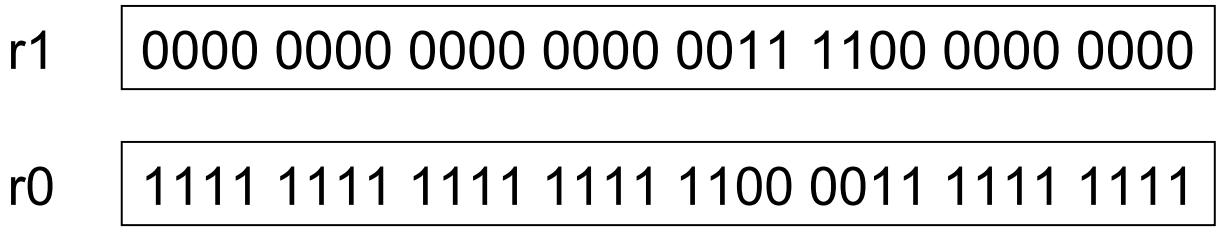


Operacja NOT

- Używana do negacji bitów słowa
  - Zmień 0 na 1 i 1 na 0

```
MVN r0, r1
```

- `r0 ≈ ~r1  (move negated)`



Instrukcje sterujące

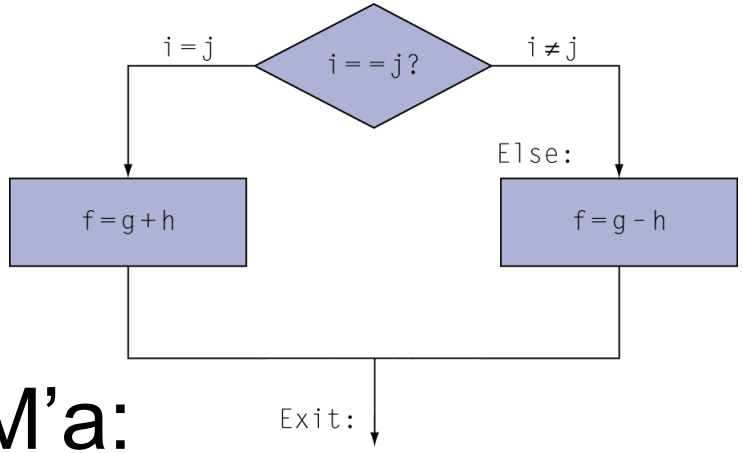
- Jeśli warunek jest słuszny, skocz do instrukcji ze wskazaną etykietą
  - Jeśli nie, przejdź dalej
- `CMP reg1, reg2`
  `BEQ L1`
  - jeśli (reg1 == reg2) skocz do etykiety L1;
- `BNE L1`
  - Odwrotny warunek: (reg1 != reg2)
- `B L1`
  - Bezwarunkowy skok do L1

Kompilacja instrukcji If

- Kod C:

```
if (i==j) f = g+h;
else f = g-h;
```

  - f, g, … w rejestrach



- Skompilowany kod ARM'a:

```
        CMP r3, r4
        BNE Else
                    ADD r0, r1, r2
        B   Exit
Else:   SUB r0, r1, r2
Exit:   …
```

Asembler oblicza adresy

Kompilacja pętli

- Kod C:

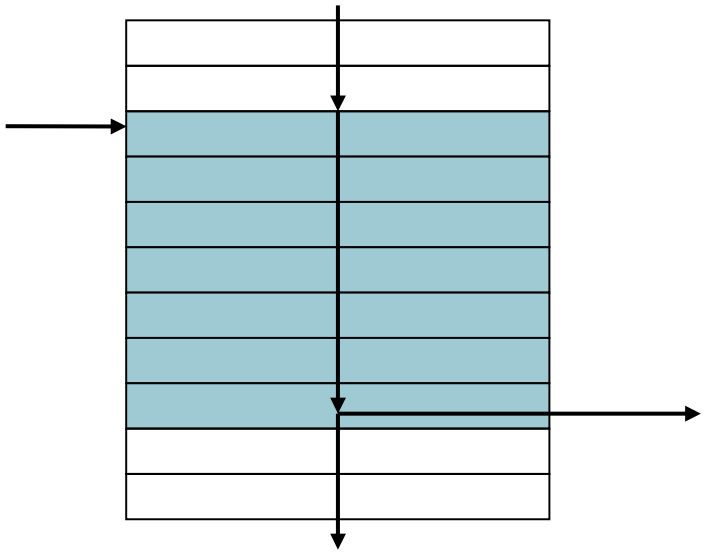
```
while (save[i] == k) i += 1;
```

  - i w r3, k w r5, adres tablicy save w r6
- Skompilowany kod ARM'a:

```
Loop: ADD   r12, r6, r3, LSL #2
      LDR   r0, [r12,#0]
      CMP   r0, r5
      BNE   Exit
      ADD   r3, r3, #1
      B     Loop
Exit: …
```

Blok instrukcji (Basic Block)

- Blok instrukcji jest ciągiem instrukcji w którym
  - Nie ma skoków (z wyjątkiem końca)
  - Nie ma etykiet (z wyjątkiem początku)



- Kompilator rozpoznaje i optymalizuje bloki instrukcji
- Zaawansowany procesor może przyspieszać ich wykonywanie

Ze znakiem – bez znaku

- Skok dla porównania ze znakiem: BLT
- Skok dla porównania bez znaku: BL0
- Przykład
  - r0 = 1111 1111 1111 1111 1111 1111 1111 1111
  - r1 = 0000 0000 0000 0000 0000 0000 0000 0001
  - ```
    CMP  r0, r1              BLTL2      ;
    skok „znakowy”
    BL0  L1       ; bez znaku
    ```
    ```

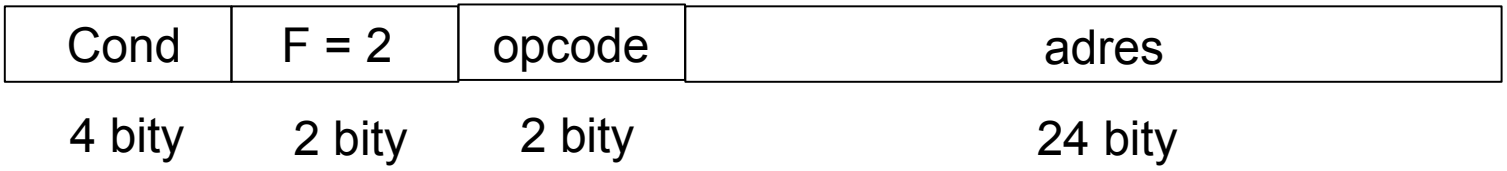
Więcej o skokach i warunkach

- Istnieje 15 wersji skoków warunkowych
  - LT: lower than, LE: lower equal
  - GT: greater than, GE: greater equal
- Instrukcje skoków rozpoznają znaczniki (condition flags) ustawiane przez CMP
- Wszystkie instrukcje DP ustawiają te znaczniki, o ile pole S jest równe 1
- W asemblerze wygląda to tak:

```
SUBS r0, r1, r2  ; ustawia znaczniki
SUB  r0, r1, r2  ; nie ustawia
```

Więcej o skokach i warunkach

- PC jest zarazem rejestrem r15!
- Format instrukcji skoku (BR)



- Adres skoku: PC + adres (ma znak)
- Cond – warunek wykonania instrukcji, KAŻDEJ z tym polem. Przykład z if:

```
CMP   r3, r4                    ADDEQ r0, r1,
r2              SUBNE r0, r1, r2
```

Adresowanie skoków

- Przykład pętli, powtórka
  - Załóżmy, że pętla po adresem 80000

```
Loop:
ADD r12, r6, r3, LSL #2
LDR r0, [r12,#0]
CMP r0, r5
BNE Exit
ADD r3, r3, #1
B   Loop
Exit: …
```

	Hex:							
80000	14	0	0	4	0	6	12	103
80004	14	1	1			12	0	000
80008	14	0	0	10	1	0	0	005
80012	1	2	2	2				
80016	14	0	1	4	0	3	3	001
80020	14	2	2	-6				
80024	…							

Wywołanie funkcji

- Kroki
  1. Umieść parametry w rejestrach
  2. Przekaż sterowanie do funkcji
  3. Zapewnij pamięć dla funkcji
  4. Wykonaj operacje funkcji
  5. Umieść wyniki w rejestrach
  6. Powróć do miejsca wywołania

Użycie rejestrów

- r0 – r3: argumenty
- r0, r1: wyniki funkcji
- r0 – r3 i r12: wartości tymczasowe
  - Mogą być zmienione przez funkcję
- r4 – r11: wartości chronione
  - Muszą być odtworzone przez funkcję
- sp: wskaźnik stosu (stack pointer, r13)
- lr: adres powrotny (link register, r14)

Wywołanie i powrót z funkcji

- Wywołanie funkcji: skok z linkiem (BR)

  `BL ProcedureLabel`

  - Adres następnej instrukcji chowany jest w lr
  - Następuje skok do adresu funkcji
- Powrót z funkcji: lr kopiowane jest do pc

  `MOV pc, lr   (= MOV r15, r14)`

  - MOV – przesyłanie zawartości rejestrów
  - Do PC można wysłać dowolny rejestr!
  - Do wykorzystania przy obliczanych skokach
    - np. w instrukcjach case/switch

Przykład „Leaf Procedure”

- Kod C:

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Argumenty g, …, j w r0, …, r3
  - f w r4 (stąd konieczność zachowania r4 na stosie)
  - Rezultat w r0

Przykład „Leaf Procedure”

- Kod ARM’a:

```
leaf_example:
   SUB  sp, sp, #4
   STR  r4, [sp,#0]
   ADD  r12, r0, r1
   ADD  r2, r2, r3
   SUB  r4, r12, r2
   MOV  r0, r4
   LDR  r4, [sp,#0]
   ADD  sp, sp, #4
   MOV  pc, lr
```

Zachowaj r4 na stosie

Ciało funkcji

Wynik

Odtwórz r4

Powrót (Return)

Funkcje „Non-Leaf”

- Funkcje wywołujące inne funkcje
- Przy zagnieżdżonym wywołaniu funkcja wywołująca musi zachować na stosie:
  - Adres powrotny
  - Wszystkie argumenty i wartości tymczasowe potrzebne po powrocie z funkcji podrzędnej
- Odtworzyć te wartości ze stosu po powrocie z funkcji podrzędnej

Przykład funkcji „Non-Leaf”

- Kod C:

```
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

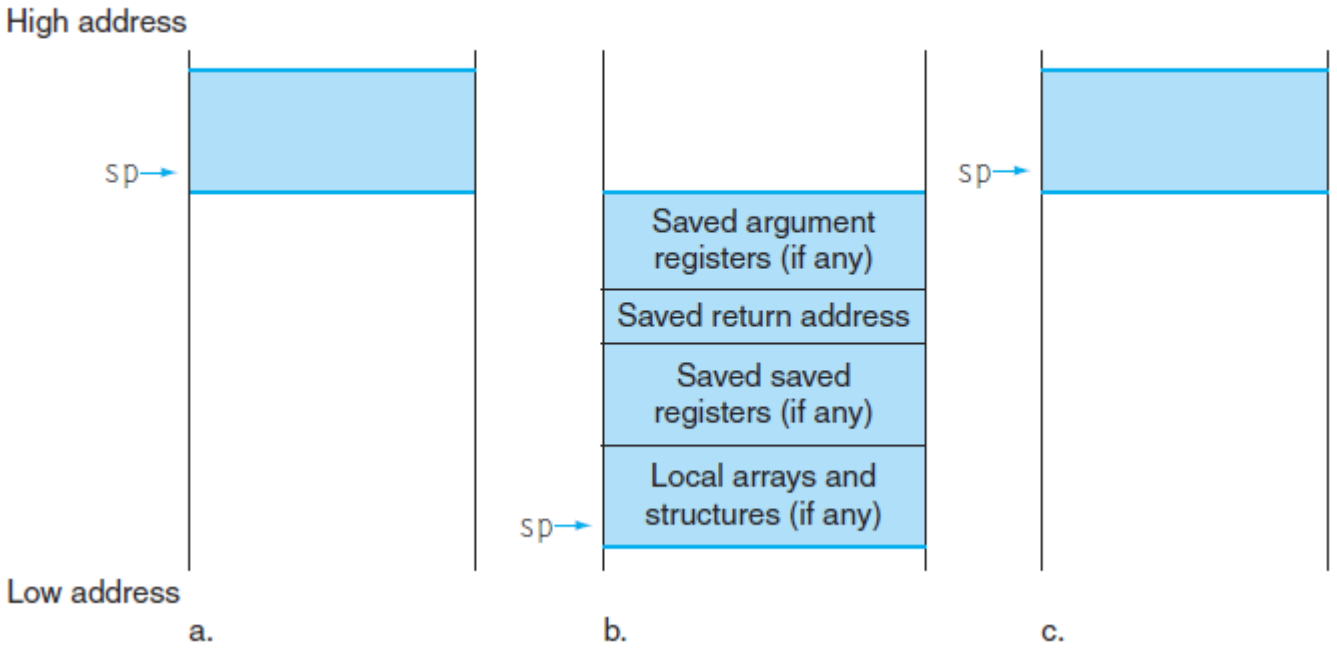
  - Argument n w r0
  - Wynik w r0

Przykład funkcji „Non-Leaf”

- Kod ARM’a:

```
fact:
    SUB  sp, sp, #8        ; przygotuj stos na 2 wartości
    STR  lr, [sp,#4]       ; zachowaj adres powrotny
    STR  r0, [sp,#0]       ; zachowaj argument
    CMP  r0, #1            ; porównaj n do 1
    BGE  L1                ; jeśli n >= 1 skocz do L1
    MOV  r0, #1            ; jeśli nie, to wynik jest 1
    ADD  sp, sp, #8        ; wyrzuć 2 wartości ze stosu
    MOV  pc, lr            ;    i wyjdź z funkcji
L1: SUB  r0, r0, #1        ; pomniejsz n
    BL   fact              ; wywołanie rekursywne
    MOV  r12, r0           ; zachowaj wynik fact
       LDR  r0, [sp,#0]      ; odzyskaj oryginalne n
    LDR  lr, [sp,#4]       ;    i adres powrotny
    ADD  sp, sp, #8        ; wyrzuć 2 wartości ze stosu
    MUL  r0, r0, r12       ; mnóż w celu uzyskania wyniku
    MOV  pc, lr            ;    i wyjdź z funkcji
```

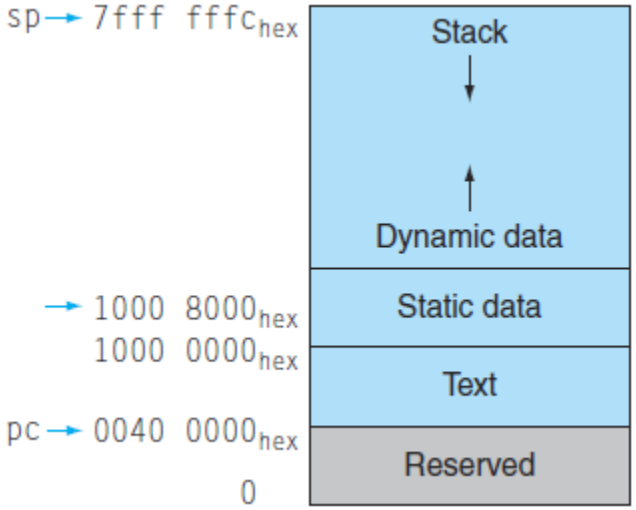
Dane lokalne na stosie



- Dane lokalne zachowane przez funkcję wywołującą
  - np. zmienne automatyczne C
  - a, b, c: stan stosu przed, podczas i po wykonaniu funkcji

Alokacja pamięci

- Tekst: kod programu
- Dane statyczne: zmienne globalne
  - np. zmienne static w C, stałe tablice i łańcuchy
- Dane dynamiczne: sterta (heap)
  - np. malloc w C, new w Javie
- Stos: pamięć automatyczna



Dane znakowe

- Zbiory znaków kodowanych bajtowo
  - ASCII: 128 znaków
    - 95 graficznych, 33 sterujących
  - Latin-1: 256 znaków
    - ASCII, +96 więcej znaków graficznych
- Unicode: zbiór znaków 32-bitowych
  - Używany w Javie, szerokie znaki C++, …
  - Większość alfabetów świata, plus symbole
  - UTF-8, UTF-16: kodowanie zmiennej długości

Operacje na bajtach

- Można używać operacje logiczne
- Ładowanie bajtu do rejestru w ARM'ie
  - Najczęściej wykorzystywane w przetwarzaniu łańcuchów

`LDRSB rt, [rs,#offset]`

  - Znak rozszerzony do 32 bitów w rt

`LDRB rt, [rs,#offset]`

  - Słowo uzupełnione zerami w rt

`STRB rt, [rs,#offset]`

  - Chowanie bajtu do pamięci (bez rozszerzania)

Kopiowanie łańcucha

- Kod C (naiwny):
  - Łańcuch kończący się znakiem null

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

  - Adresy x, y w r0, r1
  - i w r4

Kopiowanie łańcucha

- Kod ARM'a:

```
strcpy:
    SUB   sp, sp, #4      ; przygotuj stos na 1 wartość
    STR   r4, [sp,#0]     ; zachowaj r4
    MOV   r4, #0          ; i = 0
L1: ADD   r2, r4, r1      ; adres y[i] w r2
    LDRBS r3, [r2,#0]     ; r3 = y[i]
    ADD   r12, r4, r0     ; adres x[i] w r12
    STRB  r3, [r12,#0]    ; x[i] = y[i]
    BEQ   L2              ; wyjście z pętli gdy y[i] == 0

    ADD   r4, r4, 1       ; i = i + 1
    B     L1              ; następna iteracja pętli
L2: LDR   r4, [sp,#0]     ; odtworzenie r4
    ADD   sp, sp, #4      ; wyrzuć 1 wartość ze stosu
    MOV   pc, lr          ;    i wyjdź z funkcji
```

32-bitowe stałe

- Większość stałych jest mała
- Duże stałe trzeba składać, np. $abcd7654_{16}$:

```
   MOV  r0,     #&00000054
ORR r0, r0, #&00007600    ORR
r0, r0, #&00cd0000    ORR r0,
r0, #&ab000000
```

  - Pole Operand2 instrukcji DP: 4 lewe bity – przesunięcie (i), 8 – operand (X): $X \cdot 2^{2i}$
  - ARMv6T2 ma ładowanie natychmiastowe połowy słowa: MOVT (T – top), MOVW

Inne tryby adresowania (DT)

- Register offset: adres = suma rejestrów

  LDR r2, [r0,r1]
- Scaled register offset: jw. + przesunięcie

  LDR r2, [r0,r1, LSL #2]
- Immediate pre-indexed: po zaadresowaniu rejestr bazowy zapamiętuje adres:

  LDR r2, [r0, #4]!

  LDR r2, [r0, #-4]!

Inne tryby adresowania (DT)

- Immediate post-index: adres z rejestru bazowego, po czym rejestr się zmienia

  LDR r2, [r0], #4
- Register pre-index i post-index: jw., tylko rejestr z rejestrem:

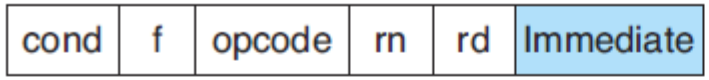
  LDR r2, [r0,r1]!

  LDR r2, [r0], r1
- Scaled register pre-indexed:

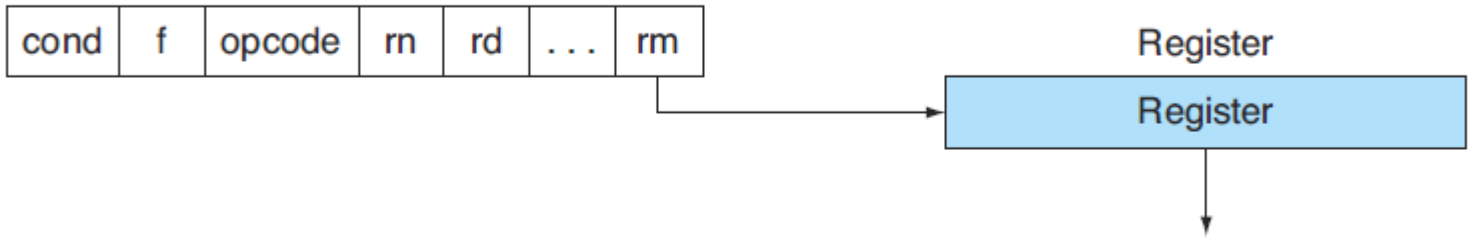
  LDR r2, [r0, r1, LSL #2]!

Podsumowanie adresowania

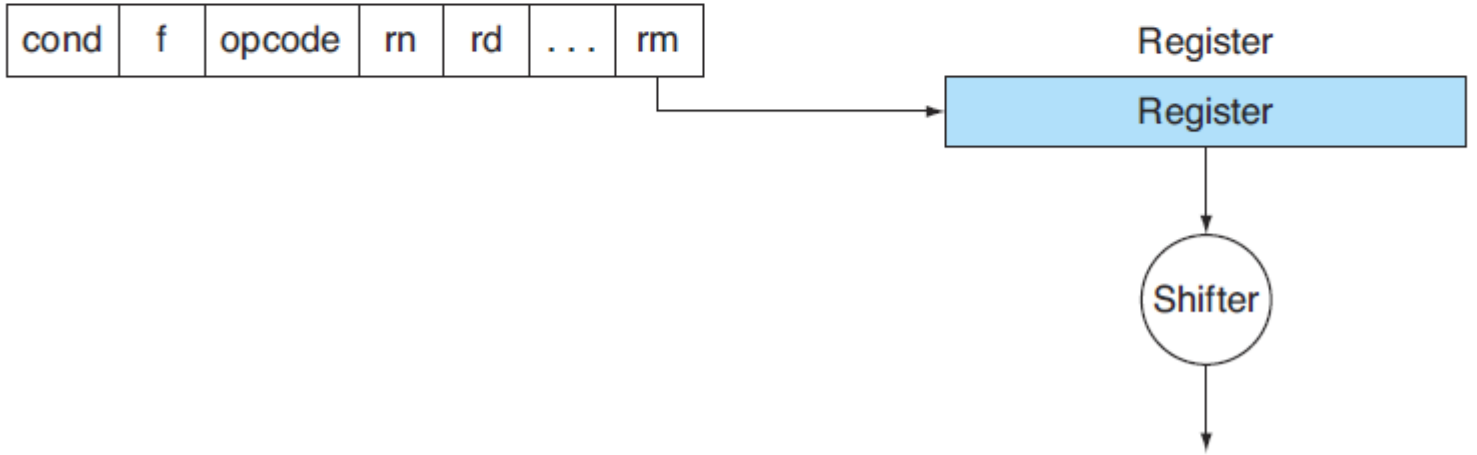
1. Immediate: ADD r2, r0, #5



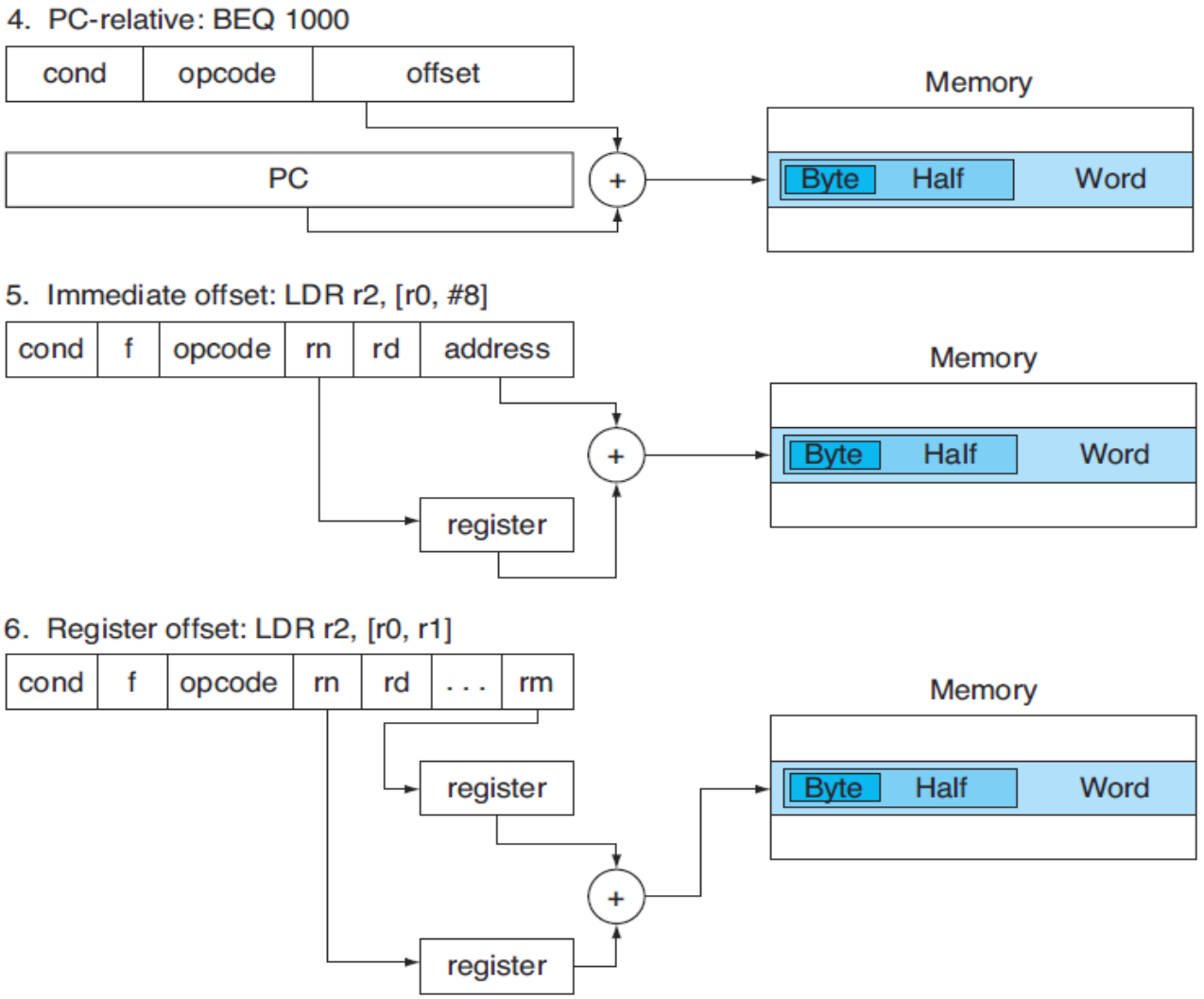
2. Register: ADD r2, r0, r1



3. Scaled register: ADD r2, r0, r1, LSL #2

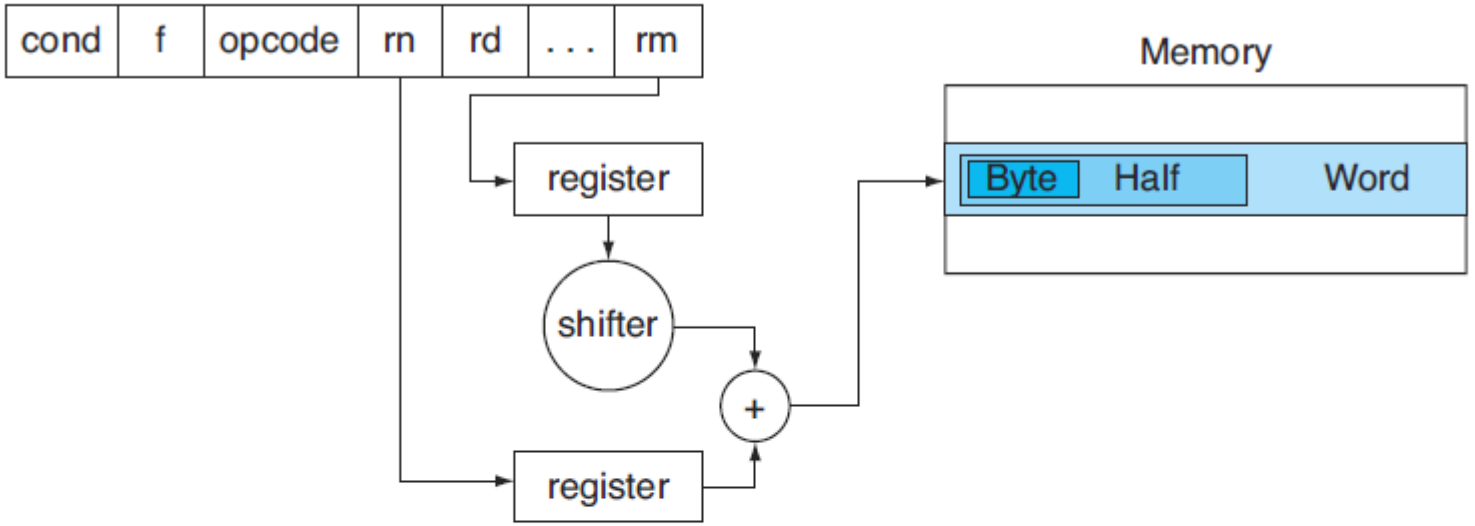


Podsumowanie adresowania

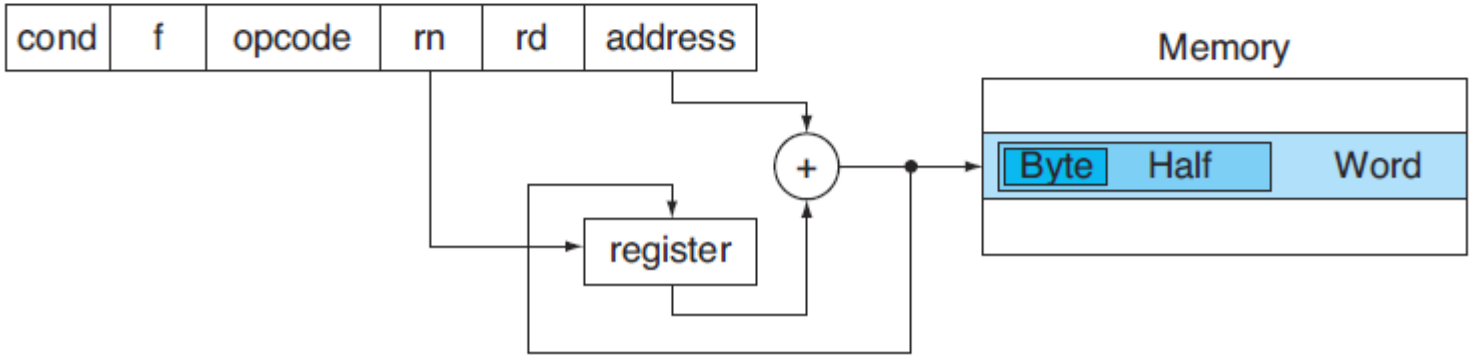


Podsumowanie adresowania

7. Scaled register offset: LDR r2, [r0, r1, LSL #2]

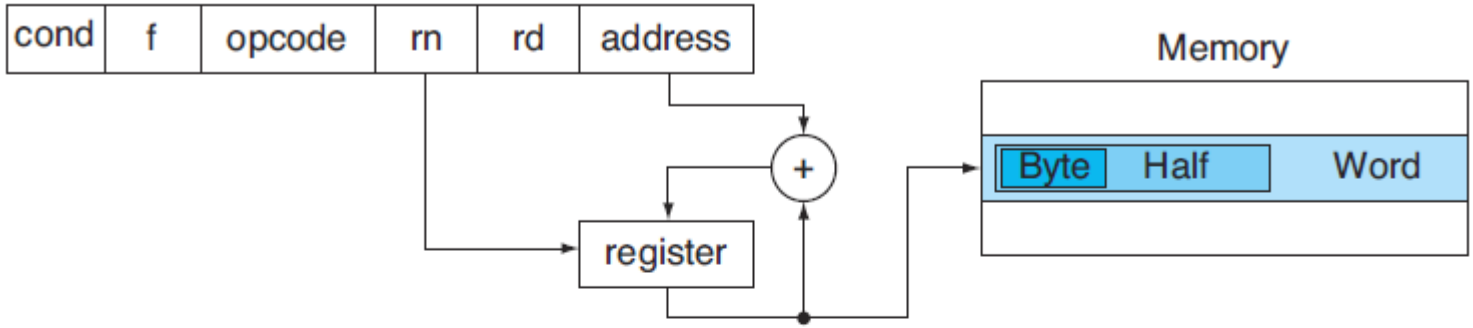


8. Immediate offset pre-indexed: LDR r2, [r0, #4]!

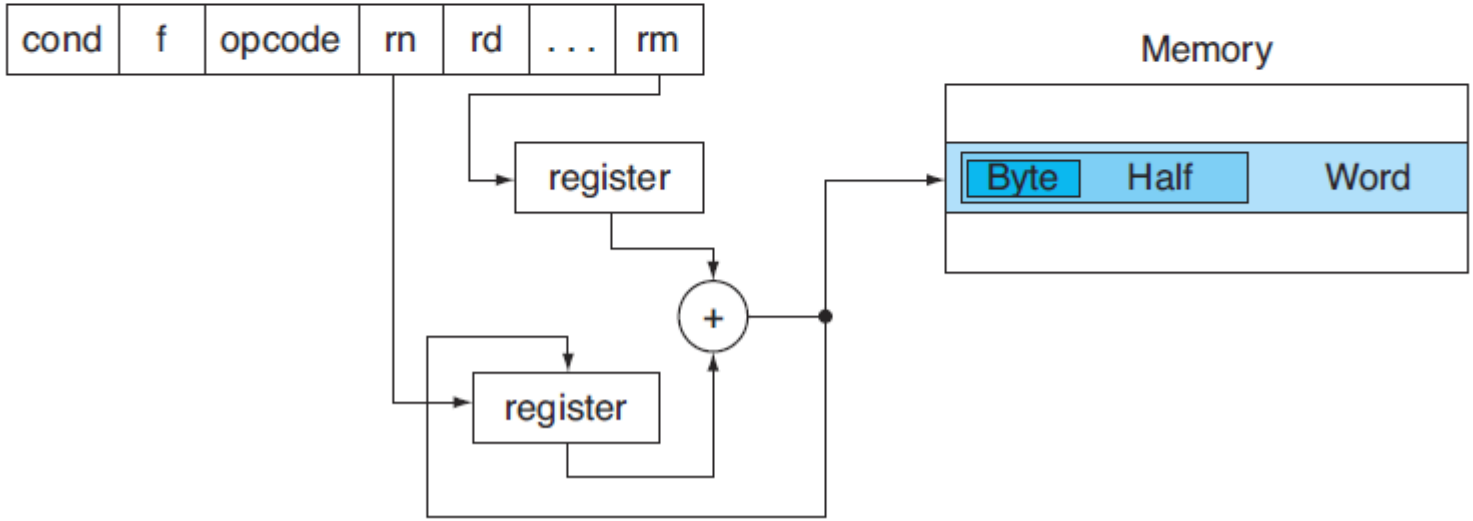


Podsumowanie adresowania

9. Immediate offset post-indexed: LDR r2, [r0], #4

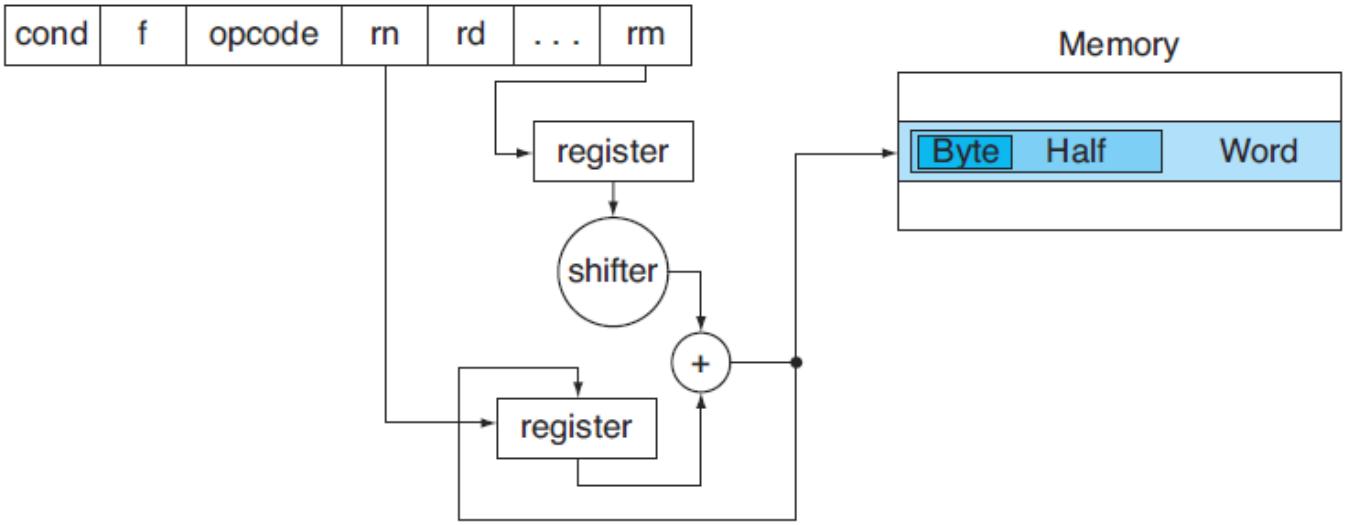


10. Register offset pre-indexed: LDR r2, [r0, r1]!

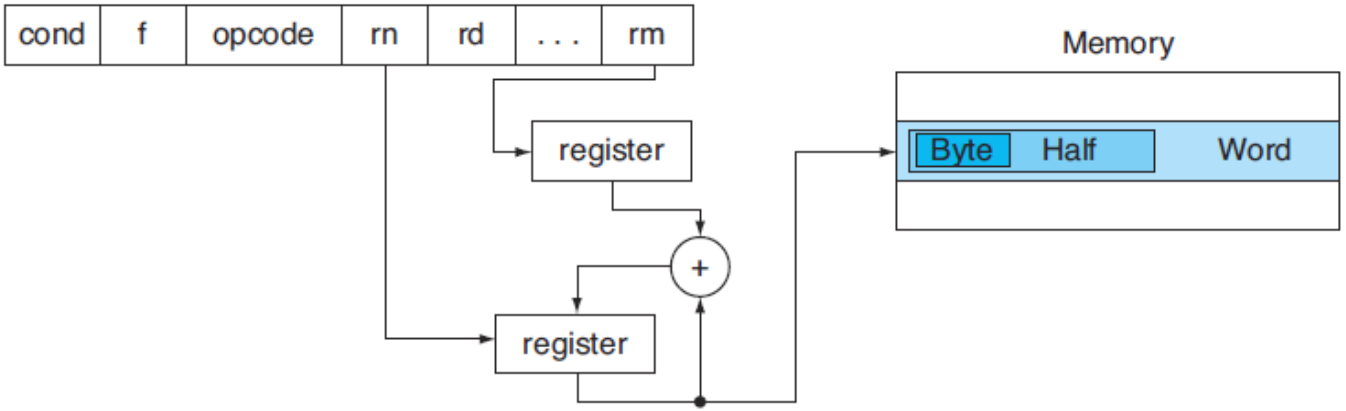


Podsumowanie adresowania

11. Scaled register offset pre-indexed: LDR r2, [r0, r1, LSL #2]!



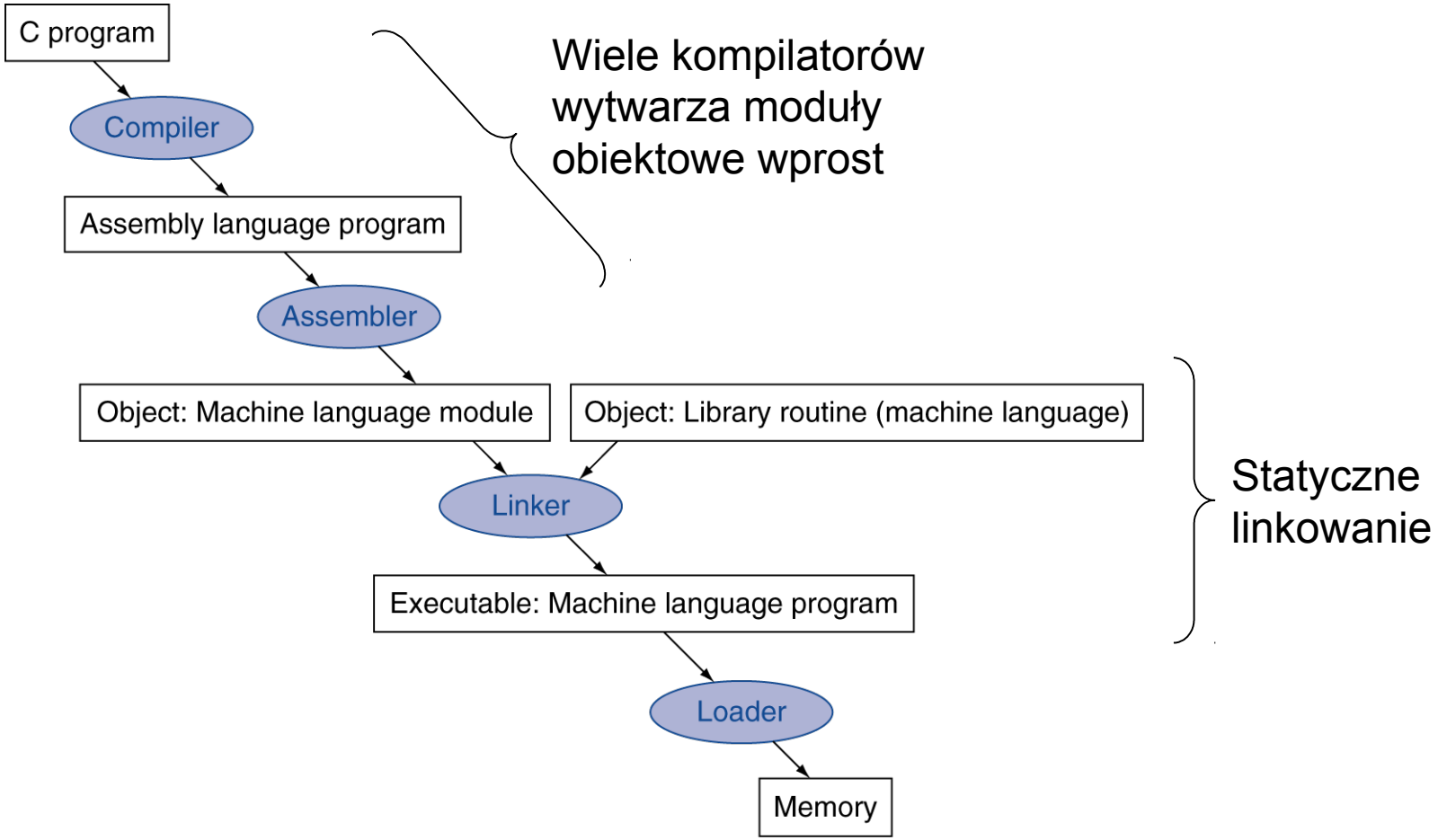
12. Register offset post-indexed: LDR r2, [r0], r1



Synchronizcja

- Dwa procesy dzielą obszar pamięci
  - P1 zapisuje, po czym P2 czyta
  - Wyścig (data race) jeśli nie są zsynchronizowane
    - Wynik zależy od kolejności dostępu
- Konieczne wsparcie sprzętowe
  - Niepodzielna (atomic) operacja odczytu/zapisu pam.
  - Brak zgody na obcy dostęp do miejsca w pamięci pomiędzy zapisem i odczytem
- Może to być jedna instrukcja
  - Np. „atomic swap” (wymiana) rejestr ↔ pamięć
  - Albo para instrukcji atomowych
  - W ARM’ie – swap: SWP rd, rm, [rn]

Translacja i start



Pseudoinstrukcje asemblera

- Większość instrukcji asemblera wprost reprezentuje instrukcje maszynowe
- Pseudoinstrukcje: warianty instrukcji ułatwiające pisanie programu, np.

```
LDR  r0, #constant
```

  - Asembler zdecyduje jaka instrukcja (instrukcje) zostanie podstawiona

Tworzenie modułu obiektów

- Asembler (lub kompilator) tłumaczy program na instrukcje maszynowe
- Dostarcza informacji niezbędnej do utworzenia z części kompletnego programu
  - Nagłówek: opisuje moduł obiektów (object module)
  - Segment tekstowy: przetłumaczone instrukcje
  - Segment danych statycznych: przypisane programowi dane niezbędne do jego funkcjonowania
  - Informacja o relokacji: dla treści zależących od położenia (absolute location) załadowanego programu
  - Tablica symboli (Symbol table): definicje globalne i referencje zewnętrzne
  - Informacja dla debugera: do kojarzenia z kodem źródłowym

Łączenie modułów obiektów

- Wytwarzany jest zbiór (obraz) wykonywalny (executable image, file)
  1. Łączone są segmenty
  2. Określane etykiety (ich adresy)
  3. Dowiązane referencje: zależne od położenia i zewnętrzne
- Zależności między lokacjami mogą być uściślane przez „relocating loader”
  - Nie ma takiej potrzeby przy pamięci wirtualnej
  - Program można ładować do miejsca (absolute location) w przestrzeni pamięci wirtualnej

Ładowanie programu

- Ładowanie z obrazu (image file) na dysku do pamięci
  1. Z nagłówka określ wymiary segmentów
  2. Utwórz wirtualną przestrzeń adresową
  3. Kopiuj tekst i inicjalizowane dane do pamięci
     - Lub ustal elementy tablicy stron (page table entry)
  4. Wstaw argumenty na stos
  5. Inicjalizuj rejestry
  6. Skocz do funkcji startowej
     - Kopiuje argumenty do r0, … i wywołuje main
     - Przy powrocie z main wykonuje „exit syscall”

Linkowanie dynamiczne

- Dołączaj funkcje biblioteczne tylko przy ich wywołaniu
  - Konieczna jest przenaszalność (relocatability) kodu funkcji
  - Unika się „puchnięcia” obrazu przez statyczne dołączenie (static linking) wszystkich (też pośrednio) wskazanych bibliotek
  - Automatycznie pobiera się nowe wersje bibliotek

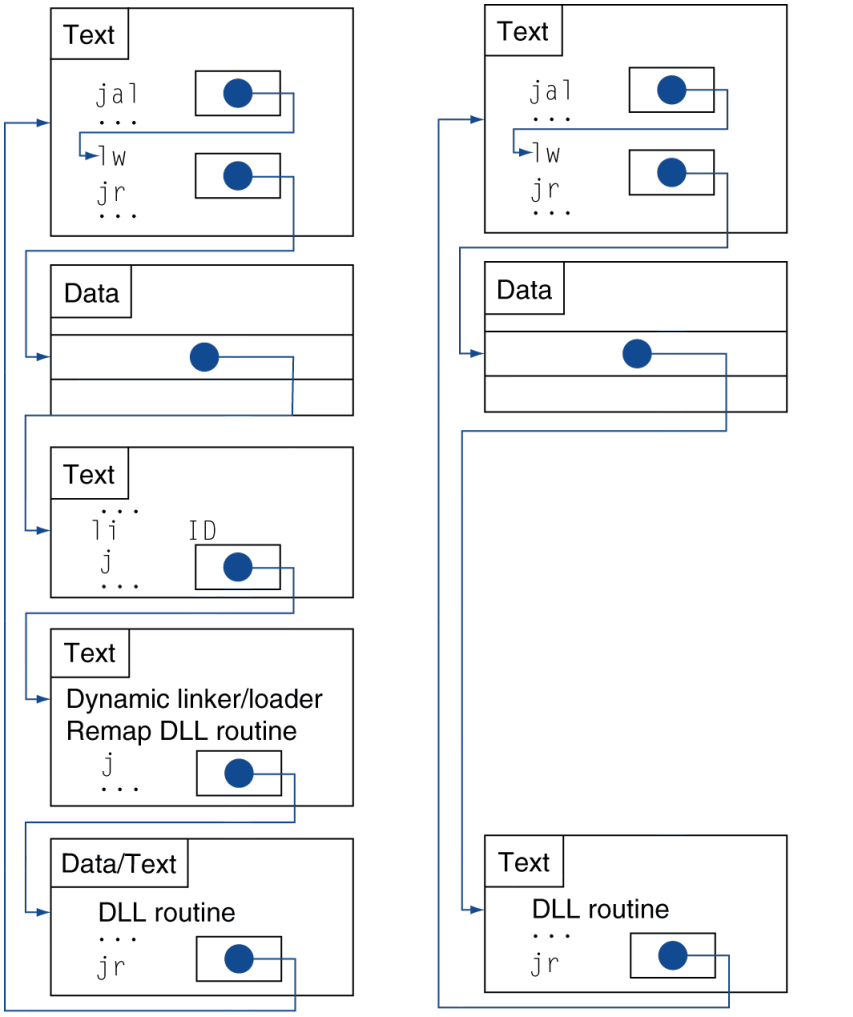
Linkowanie „leniwe” (lazy)

Tablica wskazań pośrednich (indirection table)

Pozostałość (stub): Ładuje ID funkcji, Skocz do linkera/loadera

Kod linkera/loadera

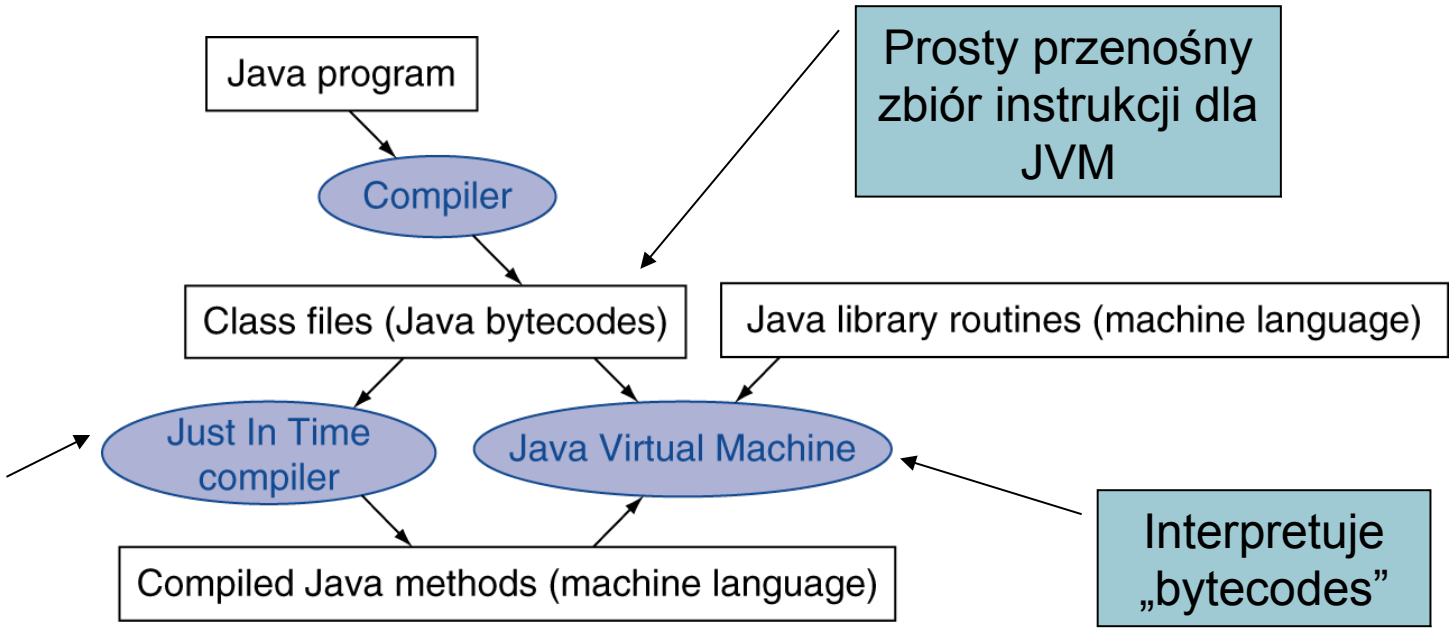
Dynamicznie odwzorowany kod



a. First call to DLL routine

b. Subsequent calls to DLL routine

Start aplikacji Java'y



Przykład: sortowanie w C

- Ilustruje użycie instrukcji asemblera w funkcji C sortowania bąbelkowego
- Funkcja swap (leaf)

```
void swap(int v[], int k)
{
  int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

  - v in r0, k in r1, temp in r2

Funkcja swap

```
swap: ADD r12, r0, r1, LSL #2
                ; r12 = v + (k*4)
                              ;   (adres v[k])
      LDR r2, [r12,#0]  ; r2 (temp) = v[k]
      LDR r3, [r12,#4]  ; r3 = v[k+1]
      STR r3, [r12,#0]  ; v[k] = r3 (v[k+1])
      STR r2, [r12,#4]  ; v[k+1] = r2 (temp)
      MOV pc, lr        ; powrót z funkcji
```

Funkcja sort w C

- Non-leaf (wywołuje swap)

```
void sort (int v[], int n)
{
  int i, j;
  for (i = 0; i < n; i += 1) {
    for (j = i – 1;
         j >= 0 && v[j] > v[j + 1];
         j -= 1) {
      swap(v,j);
    }
  }
}
```

v w r0, n w r1, i w r2, j w r3

Ciało funkcji

```
        MOV r6, r0                ; kopiuj r0 (v) do r6
        MOV r7, r1                ; kopiuj r1 (n) do r7
        MOV r2, #0                ; i = 0
for1tst: CMP r2, r1               ; porównaj i z n
        BGE exit1           ; idź do exit1 jeśli i ≥ n
        SUB r3, r2, #1            ; j = i – 1
for2tst: CMP r3, #0               ; porównaj j (r3) z zerem
        BLT exit2           ; idź do exit2 jeśli j < 0
        ADD r12, r0, r3, LSL #2 ; r12 = v + (j * 4)

        LDR r4, [r12,#0]          ; r4 = v[j]
        LDR r5, [r12,#4]          ; r5 = v[j + 1]
        CMP r4, r5          ; porównaj r4 z r5
        BLE exit2           ; idź do exit2 jeśli r5 ≥ r4
        MOV r0, r6                ; pierwszy parametr swap to v
        MOV r1, r3                ; drugi parametr swap to j
        BL  swap                  ; wywołanie funkcji swap
        SUB r3, r3, #1            ; j –= 1
        B   for2tst               ; skok do testu pętli wewnętrznej
exit2:  ADD r2, r2, #1            ; i += 1
        B   for1tst               ; skok do testu pętli zewnętrznej
```

Kopiuj parametry

Pętla zewnętrzna

Pętla wewnętrzna

Przekazanie param. i wywołanie

Pętla wewnętrzna

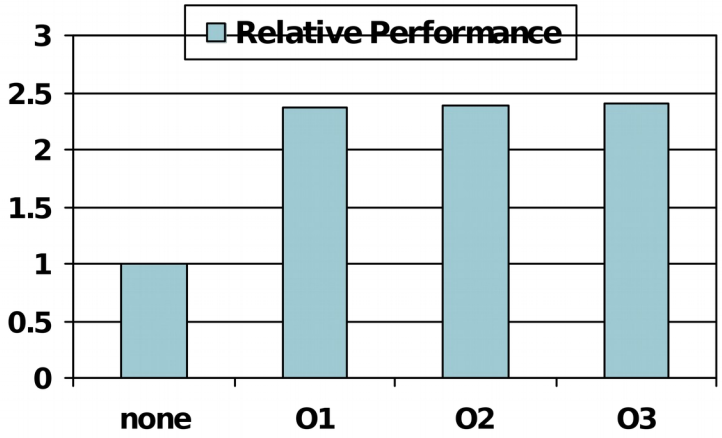
Pętla zewnętrzna

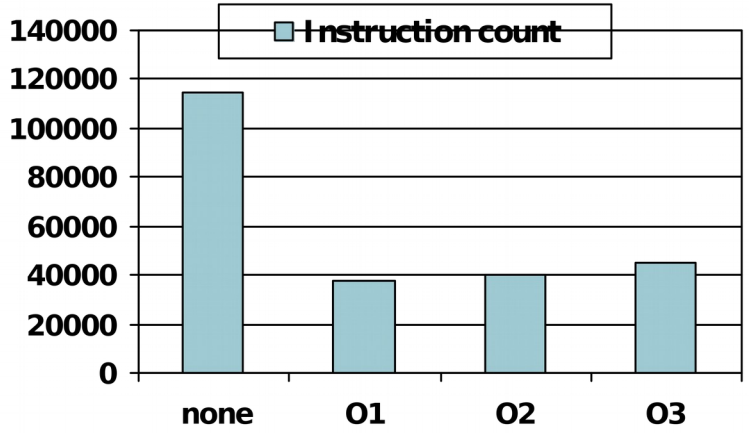
Cała funkcja

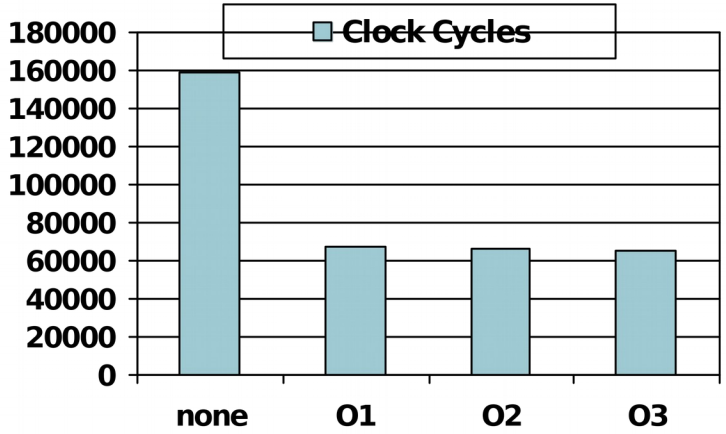
```
sort:    SUB sp, sp,#20     ; zrób miejsce na stosie na 5 rejestrów
         STR lr,[sp,#16]    ; schowaj lr na stosie
         STR r7,[sp,#12]    ; schowaj r7 na stosie
         STR r6,[sp,#8]     ; schowaj r6 na stosie
         STR r3,[sp,#4]     ; schowaj r3 (j) na stosie
         STR r2,[sp,#0]     ; schowaj r2 (i) na stosie
         …                  ; ciało funkcji
         …
  exit1: LDR r2,[sp,#0]     ; ściągnij r2 (i) ze stosu
         LDR r3,[sp,#4]     ; ściągnij r3 (j) ze stosu
         LDR r6,[sp,#8]     ; ściągnij r6 ze stosu
         LDR r7,[sp,#12]    ; ściągnij r7 ze stosu
         LDR lr,[sp,#16]    ; ściągnij lr ze stosu
         ADD sp, sp,#20     ; odtwórz wskaźnik stosu
         MOV pc, lr         ; powrót z funkcji
```

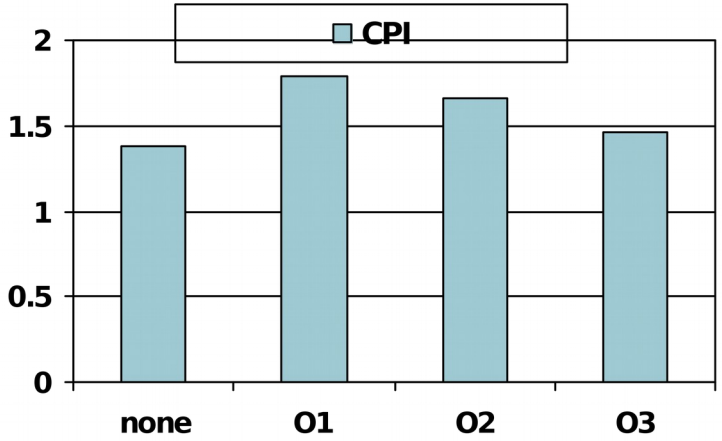
Efekt optymalizacji kompilatora

Skompilowane za pomocą gcc dla Pentium 4 pod Linuxem

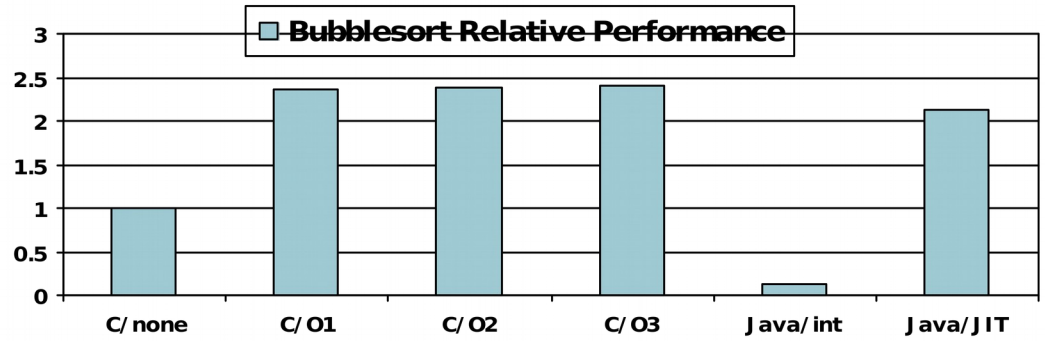


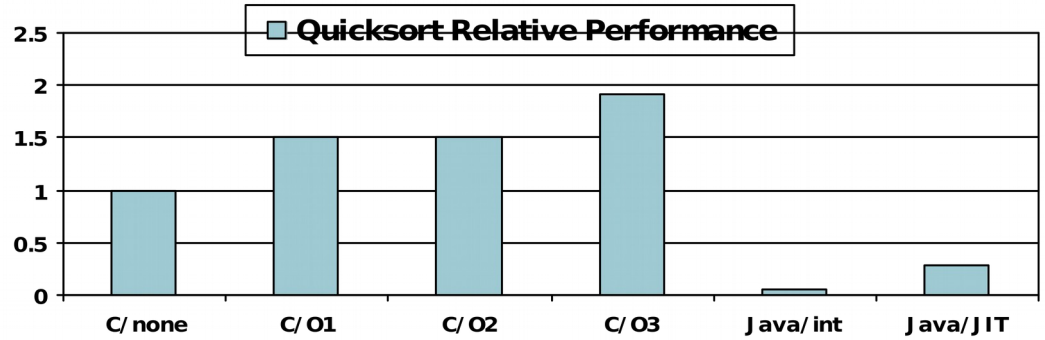


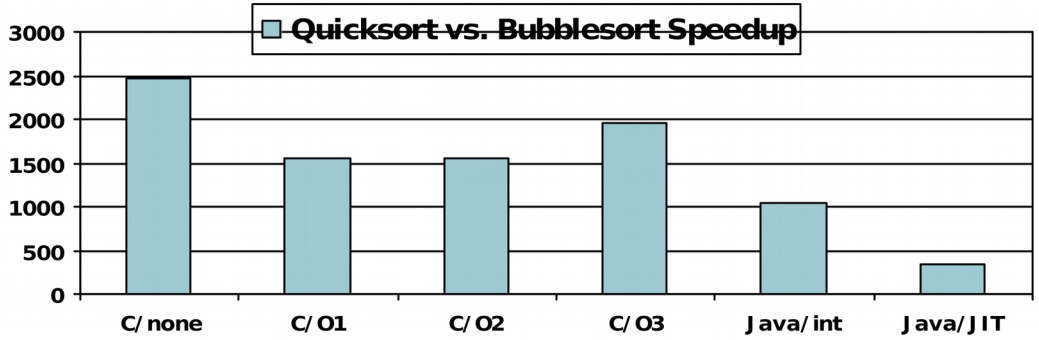




Wpływ języka i algorytmu







Wnioski

- Sama liczba instrukcji i sam CPI nie są dobrymi wskaźnikami wydajności
- Efekt optymalizacji kompilatora jest zależny od algorytmu
- Skompilowany kod Java/JIT jest istotnie szybszy niż interpretowany przez JVM
  - Nawet porównywalny do optymalnego kodu C
- Nic nie poprawi kiepskiego algorytmu!

Macierze a wskaźniki

- Indeksowanie macierzy oznacza
  - Mnożenie indeksu przez rozmiar elementu
  - Dodawanie do bazowego adresu macierzy
- Wskaźniki odpowiadają wprost adresom w pamięci
  - Można uniknąć złożoności indeksowania

Przykład: zerowanie macierzy

```
clear1(int array[], int size)
{
  int i;
  for (i = 0; i < size; i += 1)
       array[i] = 0;
}
```

```
clear2(int *array, int size)
{
  int *p;
  for (p = &array[0]; p < array[size]; p = p + 1)
       *p = 0;
}
```

```
      MOV   i,#0      ; i = 0
      MOV   zero,#0   ;zero = 0

loop1: STR  zero, [array,i, LSL #2] ; array[i] = 0
      ADD   i,i,#1    ; i = i + 1
      CMP   i,size    ; i < size
      BLT   loop1     ; if (i < size) go to loop1
```

```
      MOV   p,array     ; p = & array[0]
      MOV   zero,#0     ;zero = 0
      ADD   arraySize,array,size,LSL #2
                        ; arraySize  =  &array[size]
loop2: STR  zero,[p],#4  ; Memory[p] = 0,p = p + 4
      CMP   p,arraySize; p<&array[size]
      BLT  loop2        ; if () go to loop2
```

Porównanie: macierz – wsk

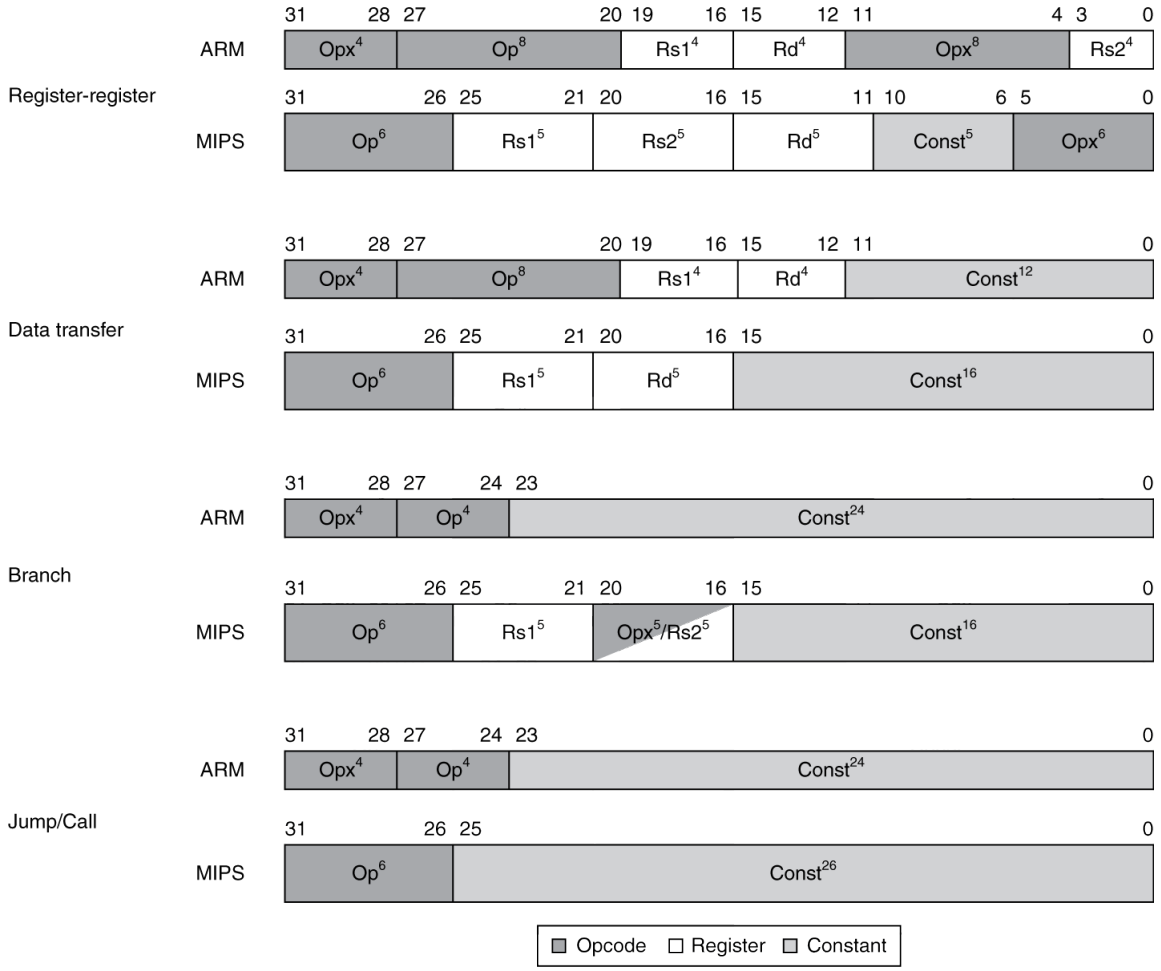
- Mnożenie zredukowane do przesunięcia
- Wersja macierzowa wymaga przesunięcia wewnątrz pętli
  - Część obliczania indeksu dla zwiększonego i
  - Porównaj – powiększanie wskaźnika
- Kompilator może uzyskać ten sam efekt jak ręczne użycie wskaźników
  - Eliminacja zmiennej indukcji
  - Lepiej pisać programy jasne i bezpieczne

Podobieństwa ARM i MIPS

- ARM: najpopularniejszy procesor zagnieżdżony
- Podobna podstawowa lista instrukcji do MIPS

	ARM	MIPS
Data ogłoszenia	1985	1985
Słowo instrukcji	32 bity	32 bity
Przestrzeń adresowa	32-bity	32-bity
Data alignment (wyrównanie słów)	Aligned	Aligned
Tryby adresowania: dane	9	3
Rejestry	15 × 32-bitowe	31 × 32-bitowe
Wejście/wyjście	Memory mapped	Memory mapped

Kodowanie instrukcji



The Intel x86 ISA

- Evolution with backward compatibility
  - 8080 (1974): 8-bit microprocessor
    - Accumulator, plus 3 index-register pairs
  - 8086 (1978): 16-bit extension to 8080
    - Complex instruction set (CISC)
  - 8087 (1980): floating-point coprocessor
    - Adds FP instructions and register stack
  - 80286 (1982): 24-bit addresses, MMU
    - Segmented memory mapping and protection
  - 80386 (1985): 32-bit extension (now IA-32)
    - Additional addressing modes and operations
    - Paged memory mapping as well as segments

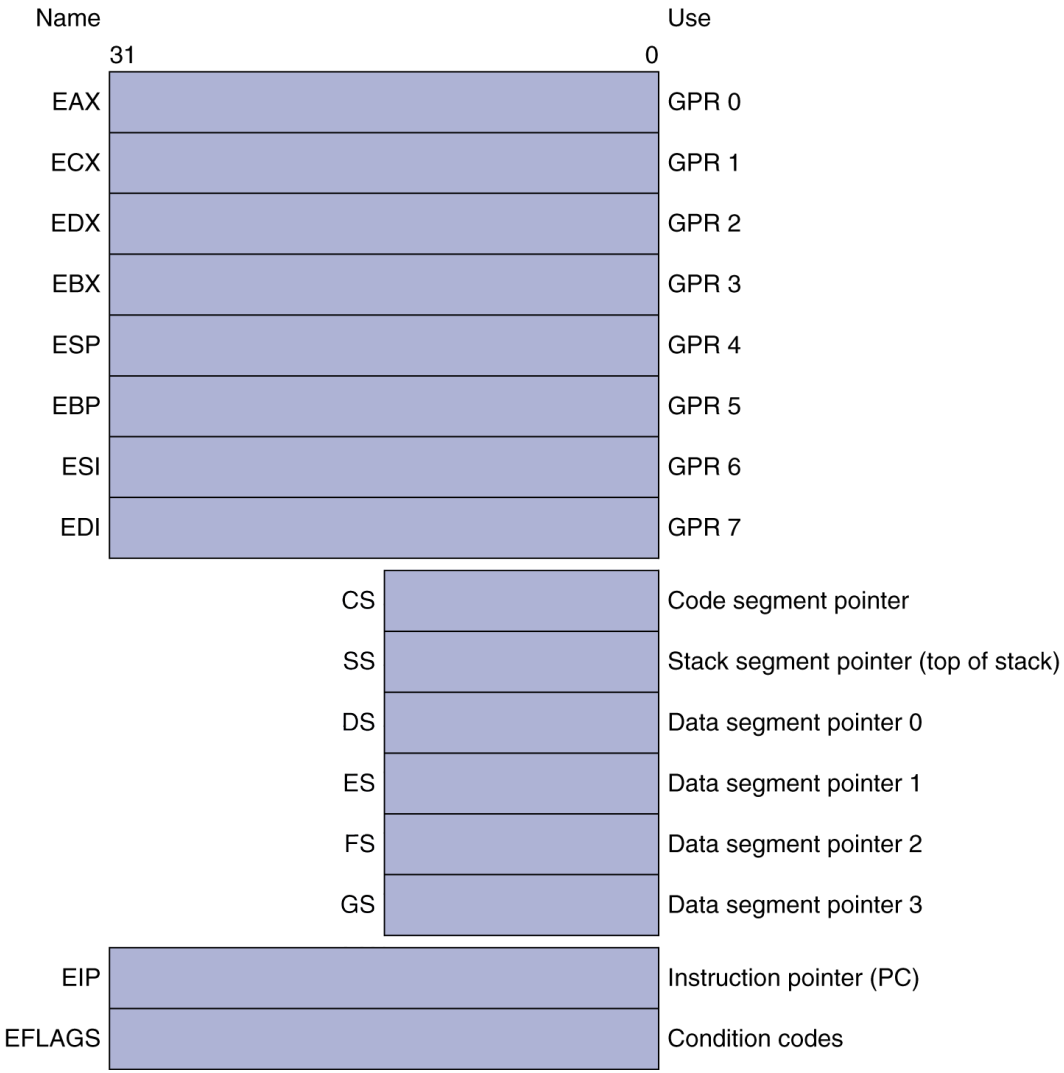
The Intel x86 ISA

- Further evolution…
  - i486 (1989): pipelined, on-chip caches and FPU
    - Compatible competitors: AMD, Cyrix, …
  - Pentium (1993): superscalar, 64-bit datapath
    - Later versions added MMX (Multi-Media eXtension) instructions
    - The infamous FDIV bug
  - Pentium Pro (1995), Pentium II (1997)
    - New microarchitecture (see Colwell, *The Pentium Chronicles*)
  - Pentium III (1999)
    - Added SSE (Streaming SIMD Extensions) and associated registers
  - Pentium 4 (2001)
    - New microarchitecture
    - Added SSE2 instructions

The Intel x86 ISA

- And further…
  - AMD64 (2003): extended architecture to 64 bits
  - EM64T – Extended Memory 64 Technology (2004)
    - AMD64 adopted by Intel (with refinements)
    - Added SSE3 instructions
  - Intel Core (2006)
    - Added SSE4 instructions, virtual machine support
  - AMD64 (announced 2007): SSE5 instructions
    - Intel declined to follow, instead…
  - Advanced Vector Extension (announced 2008)
    - Longer SSE registers, more instructions
- If Intel didn't extend with compatibility, its competitors would!
  - Technical elegance ≠ market success

Basic x86 Registers



Basic x86 Addressing Modes

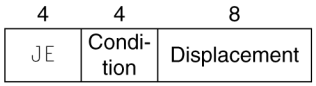
- Two operands per instruction

Source/dest operand	Second source operand
Register	Register
Register	Immediate
Register	Memory
Memory	Register
Memory	Immediate

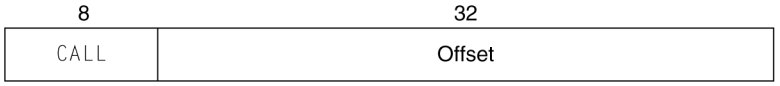
- Memory addressing modes
  - Address in register
  - Address = R_{base} + displacement
  - Address = $R_{base} + 2^{scale} \times R_{index}$ (scale = 0, 1, 2, or 3)
  - Address = $R_{base} + 2^{scale} \times R_{index}$ + displacement

x86 Instruction Encoding

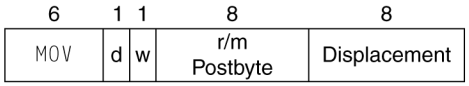
a. JE EIP + displacement



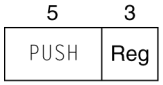
b. CALL



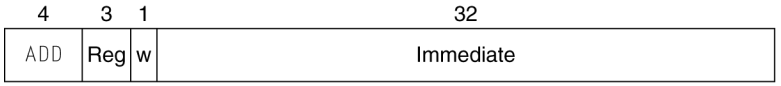
c. MOV EBX, [EDI + 45]



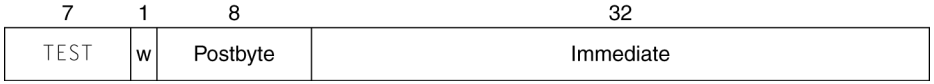
d. PUSH ESI



e. ADD EAX, #6765



f. TEST EDX, #42



- Variable length encoding
  - Postfix bytes specify addressing mode
  - Prefix bytes modify operation
    - Operand length, repetition, locking, …

Implementing IA-32

- Complex instruction set makes implementation difficult
  - Hardware translates instructions to simpler microoperations
    - Simple instructions: 1–1
    - Complex instructions: 1–many
  - Microengine similar to RISC
  - Market share makes this economically viable
- Comparable performance to RISC
  - Compilers avoid complex instructions

Instrukcje ARM v8

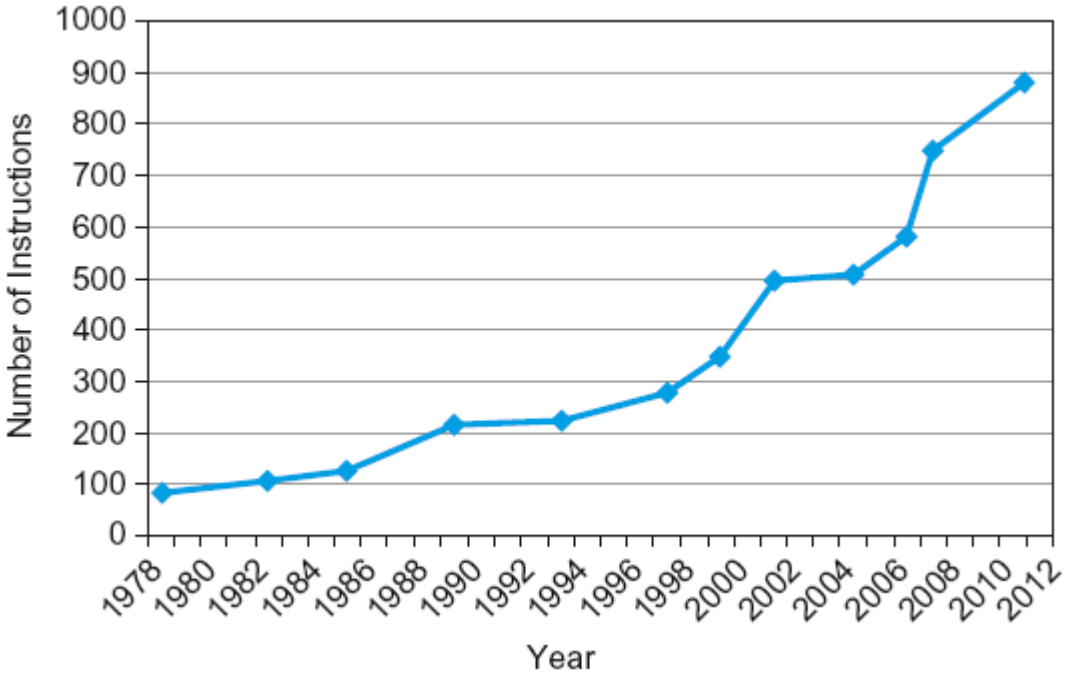
- Przechodząc do 64-bitów dokonano kompletnej przeróbki ARM
- ARM v8 przypomina MIPS
  - Zmiany w stosunku do v7:
    - Brak pola wykonania warunkowego (Cond)
    - Pole immediate jest stałą 12-bitową
    - Zrezygnowano z wielokrotnego load/store
    - PC nie jest w rejestrach ogólnego przeznaczenia (GPR)
    - Zbiór GPR rozszerzono do 32
    - Tryby adresowania działają dla wszystkich wielkości słów
    - Instrukcja dzielenia
    - Instrukcje skoku: Branch if equal/branch if not equal

Błędne poglądy

- Mocne instrukcje ↣ większa wydajność
  - Potrzeba mniej instrukcji na program
  - Ale złożone instrukcje są trudne w implementacji
    - Mogą spowolnić wszystkie instrukcje, tekże te proste
  - Kompilatory są efektywne w tworzeniu szybkiego kodu z prostych instrukcji
- Używaj asemblera dla wysokiej wydajności
  - Ale współczesne kompilatory są lepsze w radzeniu sonie z nowoczesnymi procesorami
  - Więcej linii kodu ↣ więcej błędów i mniejsza produktywność

Błedne poglądy

- Wsteczna kompatybilność ⇝ lista instrukcji nie zmienia się
  - Ale gromadzi się więcej instrukcji



Lista instrukcji x86

Pułapki

- Kolejne słowa to nie są kolejne adresy
  - Powiększaj o 4, nie o 1!
- Zachowywanie wskaźnika do zmiennej automatycznej po wyjściu z funkcji
  - np. przekazywanie wskaźnika wstecz jako argumentu
  - Wskaźnik staje się nieprawidłowy gdy zmienne (rejestry) zostaną ściągnięte ze stosu

Wnioski końcowe

- Zasady projektowania
  1. Regularność sprzyja prostocie
  2. Mniejsze jest szybsze
  3. Czyń typowy przypadek szybkim
  4. Dobry projekt wymaga dobrych kompromisów
- Warstwy oprogramowania/sprzętu
  - Kompilator, asembler, sprzęt
- ARM: jego ISA to typowy RISC
  - W przeciwieństwie do x86

Wnioski końcowe

- Liczby isntrukcji ARM'a w wykonaniach programów testowych
  - => czyń typowy przypadek szybkim
  - Weź pod uwagę kompromisy

Rodzaj instrukcji	Przykłady w ARMie	SPEC2006 Int	SPEC2006 FP
Arytmetyczne	ADD, SUB, MOV	16%	48%
Transfer danych	LDR, STR, LDRB, LDRSB, LDRH, LDRSH, STRB, STRH	35%	36%
Logiczne	AND, ORR, MNV, LSL, LSR	12%	4%
Skoki warunkowe	B_, CMP	34%	8%
Skok	B, BL	2%	0%